2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026 **Week No.: 5** **Program S. No. 10**

Title	**Online Job Market Analytics**
Problem Statement	A recruitment agency wants to analyze job opportunities posted on a dynamic job portal. Develop a Python application using Selenium to scrape job listings, extract Job Title, Company Name, Location and Salary (if available), store the collected data in a Pandas DataFrame, save the dataset as a CSV file, and visualize job demand by location.
Objectives	• To understand web scraping from a dynamic job portal using Selenium. • To extract job title, company name, location and salary information from job listings. • To organize and clean the collected data using a Pandas DataFrame. • To save the structured job dataset in CSV format. • To perform location-wise analysis and visualize job distribution using Matplotlib.
Dataset URL Link / File	**Job Portal:** https://www.naukri.com Search role used: Data Scientist | Cities: Bangalore, Hyderabad, Pune, Chennai, Mumbai and Delhi

Functions Details

Sl. No.	Function	Package	Description of function
1	Firefox()	selenium	Starts a Firefox browser session controlled by Selenium.
2	driver.get()	selenium	Opens the specified job-portal URL in the automated browser.
3	execute_script()	selenium	Executes JavaScript to scroll the dynamically loaded webpage.
4	find_elements()	selenium	Finds all matching job-card elements on the page.
5	find_element()	selenium	Finds a required field inside an individual job card.
6	time.sleep()	time	Pauses execution briefly to allow dynamic content to load.
7	DataFrame()	pandas	Converts the collected job records into tabular form.
8	drop_duplicates()	pandas	Removes duplicate job records from the DataFrame.
9	reset_index()	pandas	Resets row indexing after data cleaning.
10	to_csv()	pandas	Saves the structured job dataset as a CSV file.
11	value_counts()	pandas	Counts job records for each location for analysis.
12	bar()/savefig()	matplotlib	Creates and saves the location-wise job-demand bar chart.

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model

1. Import Selenium, Pandas, Matplotlib and supporting Python modules.
2. Define the job role, location and maximum number of job listings to collect.
3. Launch the browser using Selenium WebDriver and construct the Naukri search URL.
4. Open the dynamic job-search page and allow sufficient time for the page to load.
5. Scroll the webpage so that dynamically loaded job cards become available.
6. Locate job-card elements using CSS selectors.
7. Extract Job Title, Company Name, Location and Salary from each job card.
8. Store the extracted records in a Python list and convert them into a Pandas DataFrame.
9. Remove duplicate records and save the cleaned dataset as a CSV file.
10. Repeat the search for Bangalore, Hyderabad, Pune, Chennai, Mumbai

	and Delhi. 11. Use value_counts() to prepare location-wise job analysis and create a bar chart. 12. Save the visualization and multi-city CSV file, display the final summary and close the browser. **START -> Open Job Portal -> Scrape Listings -> DataFrame -> CSV -> Location Analysis -> Bar Chart -> STOP**
Program	(code below)

```
import time
import pandas as pd
import matplotlib.pyplot as plt
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.firefox.options import Options
from selenium.common.exceptions import NoSuchElementException

JOB_ROLE = "data scientist"
LOCATION = "bangalore"

MAX_JOBS = 30

options = Options()
driver = webdriver.Firefox(options=options)
driver.set_window_size(1400, 900)

def make_slug(text):
    return "-".join(text.lower().strip().split())

def get_text(card, selectors, default="Not Available"):
    for selector in selectors:
        try:
            element = card.find_element(By.CSS_SELECTOR, selector)
            text = element.text.strip()
            if text:
                return text
        except NoSuchElementException:
            pass
    return default

job_slug = make_slug(JOB_ROLE)
location_slug = make_slug(LOCATION)
url = f"https://www.naukri.com/{job_slug}-jobs-in-{location_slug}"
print("Search URL:", url)

driver.get(url)
time.sleep(5)
print("Page Title:", driver.title)
print("Current URL:", driver.current_url)
```

```
for i in range(3):
    driver.execute_script("window.scrollTo(0,
document.body.scrollHeight);")
    time.sleep(2)
driver.execute_script("window.scrollTo(0, 0);")

cards = driver.find_elements(By.CSS_SELECTOR, "div.cust-job-tuple")
print("Number of job cards found:", len(cards))

jobs = []
for card in cards[:MAX_JOBS]:
    title = get_text(card, ["a.title", "[class*='title']"])
    company = get_text(card, ["a.comp-name", ".comp-name"])
    location = get_text(card, ["span.locWdth", ".locWdth",
"[class*='location']"])
    salary = get_text(card, ["span.sal", ".sal", "[class*='salary']"],
                  default="Not Disclosed")
    jobs.append({
        "Job Title": title,
        "Company Name": company,
        "Location": location,
        "Salary": salary
    })

print("Jobs scraped:", len(jobs))
df = pd.DataFrame(jobs)
df = df.drop_duplicates().reset_index(drop=True)
print("Total unique jobs:", len(df))
print("Dataset Shape:", df.shape)
print(df.head(10))
print("Salary Information:")
print(df["Salary"].value_counts())

df.to_csv("job_market_data.csv", index=False)
print("CSV file saved successfully.")

csv_data = pd.read_csv("job_market_data.csv")
print(csv_data.head())

cities = ["bangalore", "hyderabad", "pune", "chennai", "mumbai",
"delhi"]
all_jobs = []

for city in cities:
    city_slug = make_slug(city)
    url = f"https://www.naukri.com/{job_slug}-jobs-in-{city_slug}"
    print("Scraping:", city)
```

```
    driver.get(url)


    time.sleep(4)

    city_cards = driver.find_elements(By.CSS_SELECTOR, "div.cust-job-
tuple")
    print("Jobs found:", len(city_cards))

    for card in city_cards[:10]:
        title = get_text(card, ["a.title", "[class*='title']"])
        company = get_text(card, ["a.comp-name", ".comp-name"])
        salary = get_text(card, ["span.sal", ".sal", "[class*='salary']"],
                          default="Not Disclosed")
        all_jobs.append({
            "Job Title": title,
            "Company Name": company,
            "Location": city.title(),
            "Salary": salary
        })

city_df = pd.DataFrame(all_jobs)
city_df = city_df.drop_duplicates().reset_index(drop=True)
print("Total jobs collected:", len(city_df))

location_analysis = city_df["Location"].value_counts().reset_index()
location_analysis.columns = ["Location", "Number of Jobs"]
print(location_analysis)

plt.figure(figsize=(10, 5))
plt.bar(location_analysis["Location"], location_analysis["Number of
Jobs"])
plt.title("Data Scientist Job Demand by Location")
plt.xlabel("Location")
plt.ylabel("Number of Job Listings")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("job_demand_by_location.png", dpi=300,
bbox_inches="tight")
plt.show()

city_df.to_csv("job_market_multi_city.csv", index=False)
print("Multi-city dataset saved successfully.")

print("ONLINE JOB MARKET ANALYTICS")
print("Job Role:", JOB_ROLE)
print("Total Jobs Analyzed:", len(city_df))
print("Cities Analyzed:", len(location_analysis))
```

```
print(location_analysis.to_string(index=False))

driver.quit()
print("Browser closed successfully.")
```

Output Snapshots With Explanation/ Observations

```
Firefox started successfully.
```

```
[5]: def make_slug(text):
         return "-".join(text.lower().strip().split())

     job_slug = make_slug(JOB_ROLE)
     location_slug = make_slug(LOCATION)

     url = f"https://www.naukri.com/{job_slug}-jobs-in-{location_slug}"

     print("Search URL:", url)
```

```
Search URL: https://www.naukri.com/data-scientist-jobs-in-bangalore
```

```
[6]: driver.get(url)

     time.sleep(5)

     print("Page Title:", driver.title)
     print("Current URL:", driver.current_url)
```

```
Page Title: Data Scientist Jobs In Bangalore - 7018 Data Scientist Job Vacancies
In Bangalore - Naukri.com
Current URL: https://www.naukri.com/data-scientist-jobs-in-bangalore
```

```
[7]: print("Website Title:")
     print(driver.title)
```

Fig. 1.1: Search URL and Job Portal Validation

The Selenium session generated the Naukri search URL for Data Scientist jobs in Bangalore and successfully retrieved the page title and current URL, confirming that the dynamic job portal was opened correctly.

```
         )
         time.sleep(2)

     driver.execute_script(
         "window.scrollTo(0, 0);"
     )

     print("Page scrolling completed.")
```

```
Page scrolling completed.
```

```
[9]: cards = driver.find_elements(
         By.CSS_SELECTOR,
         "div.cust-job-tuple"
     )

     print("Number of job cards found:", len(cards))
```

```
Number of job cards found: 20
```

```
[10]: print(cards[0].text)
```

```
Data Scientist
Aditya Birla Management Corporation (ABMCPL)
4.1
2366 Reviews
7-10 Yrs
Bengaluru
Position Title - Data Scientist Job Location - BengaluruJob Purpose The role
focuses o…
Machine LearningDeep LearningPythonPredictive ModelingPysparkArtificial
IntelligenceNatural Language ProcessingNeural Networks
```

Fig. 1.2: Detection of Dynamic Job Cards

After scrolling the page, Selenium detected 20 job-card elements. The first job card displays details such as job title, company, experience, location and associated skills.

```
    jobs.append({
        "Job Title": title,
        "Company Name": company,
        "Location": location,
        "Salary": salary
    })

print("Jobs scraped:", len(jobs))
```

```
Jobs scraped: 20
```

```
[14]: jobs[:3]
```

```
[14]: [{'Job Title': 'Data Scientist',
        'Company Name': 'Aditya Birla Management Corporation (ABMCPL)',
        'Location': 'Bengaluru',
        'Salary': 'Not Disclosed'},
       {'Job Title': 'Data Scientist/Software Developer',
        'Company Name': 'Xylem Water Solutions',
        'Location': 'Bengaluru',
        'Salary': 'Not Disclosed'},
       {'Job Title': 'Data Scientist III',
        'Company Name': 'Walmart',
        'Location': 'Bengaluru',
        'Salary': 'Not Disclosed'}]
```

```
[15]: df = pd.DataFrame(jobs)

      df
```

```
[15]:                                         Job Title  \
      0                                 Data Scientist
      1               Data Scientist/Software Developer
      2                             Data Scientist III
      3                                 Data Scientist
      4        Data Scientist | Python | GenAI | LLM | RAG
      5                 Machine Learning Data Scientist
      6                        Advanced Data Scientist
      7                                 Data Scientist
      8    Data Scientist | GenAI | Agentic AI | LLM | RAG
      9                            Data Scientist AIML
      10  Data Scientist/Python/MLops- 4th Sep(Fri)-6+ y…
      11                            Data scientist III
      12                                Data Scientist
      13                     Data Scientist/ Data Analyst
      14                        Walk-in || Data Scientist
```

Fig. 1.3: Scraped Job Records and Pandas DataFrame

The scraper collected 20 listings and extracted the required Job Title, Company Name, Location and Salary fields. The records were then converted into a structured Pandas DataFrame.

```
7                      Pune, Bengaluru  Not Disclosed
8   Hybrid - Pune, Bengaluru, Delhi / NCR  Not Disclosed
9            Hyderabad, Chennai, Bengaluru  Not Disclosed
```

```
[18]: print("Dataset Shape:", df.shape)

      print("\nColumn Names:")
      print(df.columns.tolist())

      print("\nTotal Records:", len(df))
```

```
Dataset Shape: (20, 4)

Column Names:
['Job Title', 'Company Name', 'Location', 'Salary']

Total Records: 20
```

```
[19]: print("Salary Information:")

      print(df["Salary"].value_counts())
```

```
Salary Information:
Salary
Not Disclosed    16
20-30 Lacs PA     1
15-20 Lacs PA     1
14-20 Lacs PA     1
0-18 Lacs PA      1
Name: count, dtype: int64
```

```
[20]: df.to_csv(
          "job_market_data.csv",
```

Fig. 1.4: Dataset Structure, Salary Availability and CSV Creation

The initial dataset contains 20 records and 4 required columns. Salary information was unavailable for most listings, while a few postings contained salary ranges. The cleaned DataFrame was saved successfully as job_market_data.csv.

```
          index=False
      )

      print("CSV file saved successfully.")
```

```
CSV file saved successfully.
```

```
[21]: csv_data = pd.read_csv(
          "job_market_data.csv"
      )

      csv_data.head()
```

```
[21]:                              Job Title  \
0                           Data Scientist
1          Data Scientist/Software Developer
2                       Data Scientist III
3                           Data Scientist
4  Data Scientist | Python | GenAI | LLM | RAG

                             Company Name  \
0  Aditya Birla Management Corporation (ABMCPL)
1                    Xylem Water Solutions
2                                  Walmart
3                                  Genpact
4                                  Infosys

                          Location         Salary
0                        Bengaluru  Not Disclosed
1                        Bengaluru  Not Disclosed
2                        Bengaluru  Not Disclosed
3                        Bengaluru  20-30 Lacs PA
```

Fig. 1.5: Verification of the Saved CSV Dataset

The generated CSV file was read back using pandas.read_csv(), and the displayed rows confirm that the job-listing data was stored correctly in structured form.

```
            "Company Name": company,
            "Location": city.title(),
            "Salary": salary
        })
```

```
Scraping: bangalore
Jobs found: 20

Scraping: hyderabad
Jobs found: 20

Scraping: pune
Jobs found: 20

Scraping: chennai
Jobs found: 20

Scraping: mumbai
Jobs found: 21

Scraping: delhi
Jobs found: 21
```

```
[25]: city_df = pd.DataFrame(all_jobs)

      city_df = city_df.drop_duplicates().reset_index(drop=True)

      print("Total jobs collected:", len(city_df))

      city_df.head(10)
```

```
Total jobs collected: 60
```

```
[25]:                                    Job Title  \
      0                              Data Scientist
      1            Data Scientist/Software Developer
      2                          Data Scientist III
      3                              Data Scientist
      4     Data Scientist | Python | GenAI | LLM | RAG
      5                Machine Learning Data Scientist
      6                     Advanced Data Scientist
      7                              Data Scientist
```

Fig. 1.6: Multi-City Job Data Collection

The same Data Scientist search was executed for Bangalore, Hyderabad, Pune, Chennai, Mumbai and Delhi. A controlled sample of 10 listings per city produced 60 records for location-wise analysis.

```
4                    Infosys  Bangalore  Not Disclosed
5                       Hsbc  Bangalore  Not Disclosed
6                 ExxonMobil  Bangalore  Not Disclosed
7                       VOIS  Bangalore  Not Disclosed
8                    Infosys  Bangalore  Not Disclosed
9  Tata Consultancy Services  Bangalore  Not Disclosed
```

```
[26]: location_analysis = (
          city_df["Location"]
          .value_counts()
          .reset_index()
      )

      location_analysis.columns = [
          "Location",
          "Number of Jobs"
      ]

      location_analysis
```

```
[26]:     Location  Number of Jobs
      0  Bangalore              10
      1  Hyderabad              10
      2       Pune              10
      3    Chennai              10
      4     Mumbai              10
      5      Delhi              10
```

```
[27]: plt.figure(figsize=(10, 5))
```

Fig. 1.7: Location-Wise Job Analysis

Pandas value_counts() produced the location-wise summary for the six-city job sample.

```
plt.tight_layout()

plt.show()
```

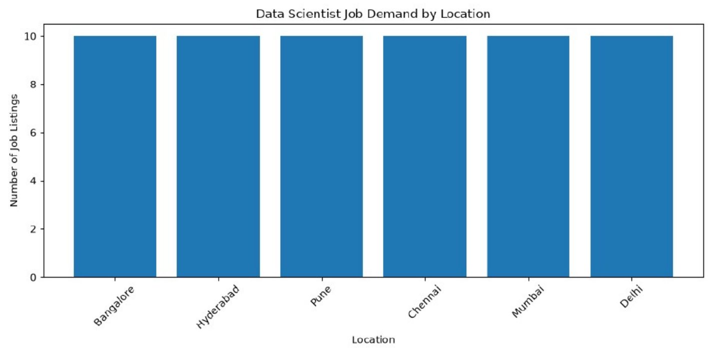

Fig. 1.8: Data Scientist Job Demand Visualization by Location

A Matplotlib bar chart visualizes the number of sampled job listings for each selected city. The graph provides a clear location-wise comparison of the collected job-market sample.

```
[30]: print("ONLINE JOB MARKET ANALYTICS")
      print("-" * 35)

      print("Job Role:", JOB_ROLE)
      print("Total Jobs Analyzed:", len(city_df))
      print("Cities Analyzed:", len(location_analysis))

      print("\nLocation-wise Job Demand:")
      print(location_analysis.to_string(index=False))
```

```
ONLINE JOB MARKET ANALYTICS
-----------------------------------
Job Role: data scientist
Total Jobs Analyzed: 60
Cities Analyzed: 6

Location-wise Job Demand:
 Location  Number of Jobs
Bangalore              10

Hyderabad              10
     Pune              10
  Chennai              10
   Mumbai              10
    Delhi              10
```

```
[31]: driver.quit()

      print("Browser closed successfully.")
```

Fig. 1.9: Final Job Market Analytics Summary

The final output confirms that the multi-city dataset and graph were saved successfully. The analysis includes 60 sampled job records across 6 cities and displays the location-wise summary.

Conclusions

The exercise successfully demonstrated Selenium-based web scraping on a dynamic job portal. Job title, company name, location and available salary information were extracted from Naukri, organized in Pandas DataFrames, cleaned and stored as CSV files. Location-wise analysis was performed for six cities and visualized using a Matplotlib bar chart. Thus, the required job listings dataset, CSV output, location analysis and demand visualization were generated successfully.

References

1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018.

2. Python Program (GitHub link): https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/Lab%20Codes/Week5-EP.ipynb

3. Class lecture link: https://drive.google.com/file/d/1HjCacfaFs49XEnuM3idduuKB0Tgjb4xI/view